Ejercicios con funciones de Python

Utiliza map(), filter(), lambda, sorted(), sum(), min(), max(), set(), len() y el método lista.count(valor), según corresponda.

No uses pandas, bucles for/while ni comprensiones en los cálculos. Puedes definir funciones auxiliares con def y utilizar condiciones, índices y cortes de listas. Muestra cada resultado con un nombre claro.

```
# Cada tupla: (nombre, horas, puntos, entregas, minutos)
alumnos = [
    ("Ana",      2,  50,  4, 30),
    ("Bruno",    3,  60,  5, 35),
    ("Carla",    4,  60,  6, 40),
    ("Diego",    5,  70,  7, 45),
    ("Elena",    5,  70,  7, 50),
    ("Fátima",   6,  80,  8, 55),
    ("Gonzalo",  6,  80,  8, 60),
    ("Hugo",     7,  80,  9, 65),
    ("Irene",    8,  90, 10, 70),
    ("Lucía",    9, 100, 10, 80),
]
```

Índices: nombre [0], horas [1], puntos [2], entregas [3], minutos [4]. Cada ejercicio parte de los datos originales. map() y filter() devuelven iteradores que se consumen al recorrerlos; guárdalos con list() antes de reutilizar sus resultados. Redondea a 2 decimales solo al mostrar resultados.

1 Limpiar y validar puntuaciones

```
datos = [" 50", "60 ", "", "NA", "80",
         "105", "-5", "70", " NA ", "90"]
```

Elimina los espacios de los extremos con strip(). Descarta las cadenas vacías y "NA" **antes** de convertir a entero. Después conserva solo los valores entre 0 y 100, ambos incluidos. Muestra la lista válida, cuántos elementos se descartaron en total y la media de los válidos; si no queda ninguno, muestra None como media.

Herramientas: map(), filter(), lambda, int(), sum(), len(); el método texto.strip() elimina espacios de los extremos.

2 Aplicar una bonificación con límite

Quienes tienen al menos 8 entregas reciben 5 puntos extra, sin superar 100. Genera una lista de **(nombre, puntos_nuevos)** para todos. Indica cuántos cumplen la condición de entregas y cuántos aumentan realmente su puntuación. Conserva intacta la lista alumnos.

Herramientas: map(), filter(), lambda, min(), len().

3 Combinar tres condiciones

Selecciona a quienes estudian **como máximo 6 horas**, obtienen **al menos 75 puntos** y tienen **8 entregas o más**. Muestra sus nombres en orden alfabético y la media de sus puntos. Si no hay seleccionados, la media debe ser None.

Herramientas: filter(), map(), lambda, sorted(), sum(), len().

4 Crear una clasificación con desempates

Ordena a todos por puntos de mayor a menor. En caso de empate, coloca antes a quien estudie menos horas; si también empatan, ordena por nombre ascendente. Muestra los cinco primeros como **(nombre, puntos, horas)**. Usa el orden natural de las cadenas de Python.

Herramientas: sorted() con key y una tupla de criterios, lambda, map(), corte [:5].

5 Calcular una media recortada

Extrae los puntos y ordénalos. Elimina las **dos puntuaciones menores y las dos mayores**, retirando exactamente cuatro datos aunque haya empates. Calcula la media de los restantes y la media original. Muestra ambas y la diferencia **media_recortada - media_original**.

Herramientas: map(), sorted(), cortes de listas, sum(), len().

6 Contar frecuencias y encontrar todas las modas

Extrae los puntos de alumnos y crea una lista nueva añadiendo un 70. Genera **un par (puntuación, frecuencia) por cada puntuación distinta** y ordénalos por frecuencia descendente; en caso de empate, por puntuación ascendente. Obtén **todas** las puntuaciones que tengan la frecuencia máxima, ordenadas de menor a mayor.

Herramientas: set(), lista.count(valor), map(), filter(), max(), sorted(), lambda.

7 Comparar puntos por hora

Genera pares **(nombre, puntos / horas)**. Calcula la media de esas diez razones individuales, no el cociente entre los puntos totales y las horas totales. Selecciona a quienes superen estrictamente esa media y ordénalos por razón descendente. Muestra sus nombres y razones. Todas las horas del conjunto son positivas.

Herramientas: map(), sum(), len(), filter(), sorted(), lambda.

8 Resumir tres tramos de puntuación

```
rangos = [("Tramo 1", 0, 69),
          ("Tramo 2", 70, 84),
          ("Tramo 3", 85, 100)]
```

Para cada tramo, obtén sus alumnos mediante filter() y devuelve **(nombre_tramo, cantidad, media_puntos)**. Incluye los límites de cada tramo. Genera la lista de resúmenes aplicando map() a rangos; puedes usar una función auxiliar. Si un tramo queda vacío, su media será None. Comprueba que las tres cantidades suman el total de alumnos.

Herramientas: map(), filter(), lambda o función auxiliar, sum(), len().

9 Medir la evolución entre dos pruebas

```
prueba2 = [55, 65, 60, 75, 80, 75, 90, 85, 90, 100]
```

prueba2 contiene las nuevas puntuaciones **en el mismo orden que alumnos**. Usa map() con alumnos y prueba2 para generar pares **(nombre, puntos_nuevos - puntos_originales)**. Cuenta cuántos mejoran, mantienen o empeoran su resultado. Obtén todos los nombres con la mejora máxima, sin perder empates.

Herramientas: map() con dos listas, lambda, filter(), len(), max().

10 Preparar un informe de un grupo

Selecciona a quienes estudian **como máximo 6 horas** y tienen **al menos 6 entregas**. Sobre ese grupo, calcula la cantidad, la media de puntos y todos los nombres con la puntuación máxima. Muestra además los participantes como **(nombre, puntos, entregas)**, ordenados por puntos descendentes y nombre ascendente. Si el grupo está vacío: cantidad 0, media None y listas vacías.

Herramientas: filter(), map(), sorted(), sum(), len(), max(), lambda.